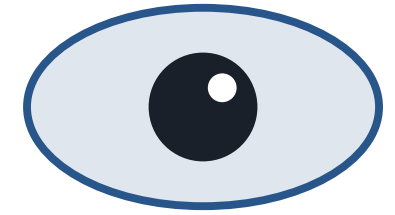


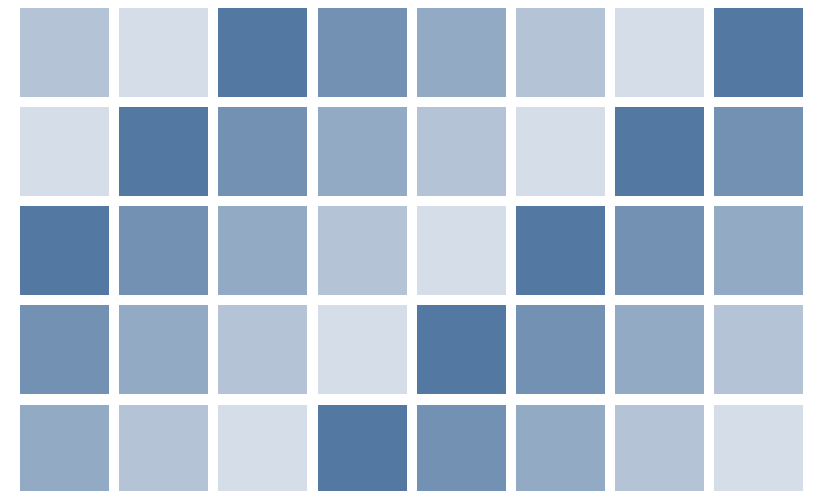
LECTURE 5



Computer Vision

Images, Colour Spaces & Filters

With practical implementation



Dr. Muhammad Kazim

Department of Intelligent Systems
University of Lahore

HUMAN

NATURAL INTELLIGENCE



Innate

Present from birth



Emotions

Feels and understands



Creativity

Imagines and innovates



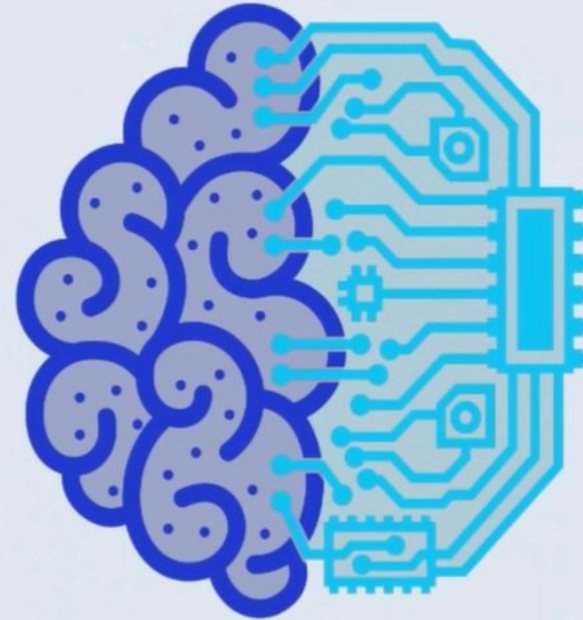
Experience

Learns from life and adapts

VS

COMPUTER

ARTIFICIAL INTELLIGENCE



Built

Created by humans



Data

Learns from data



Programming

Follows instructions



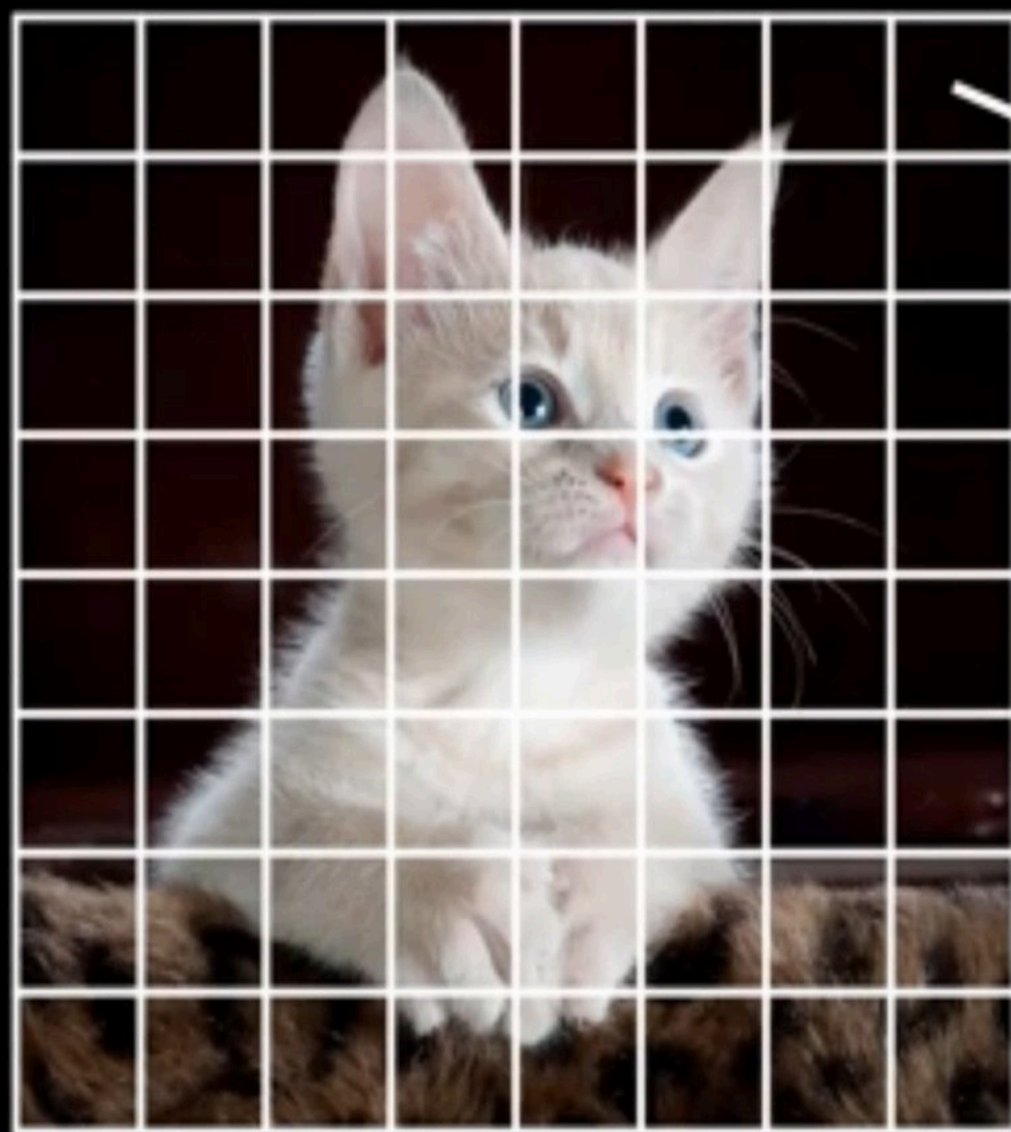
No Emotions

Does not feel or understand

Natural Intelligence is biological.
It comes from the human brain.

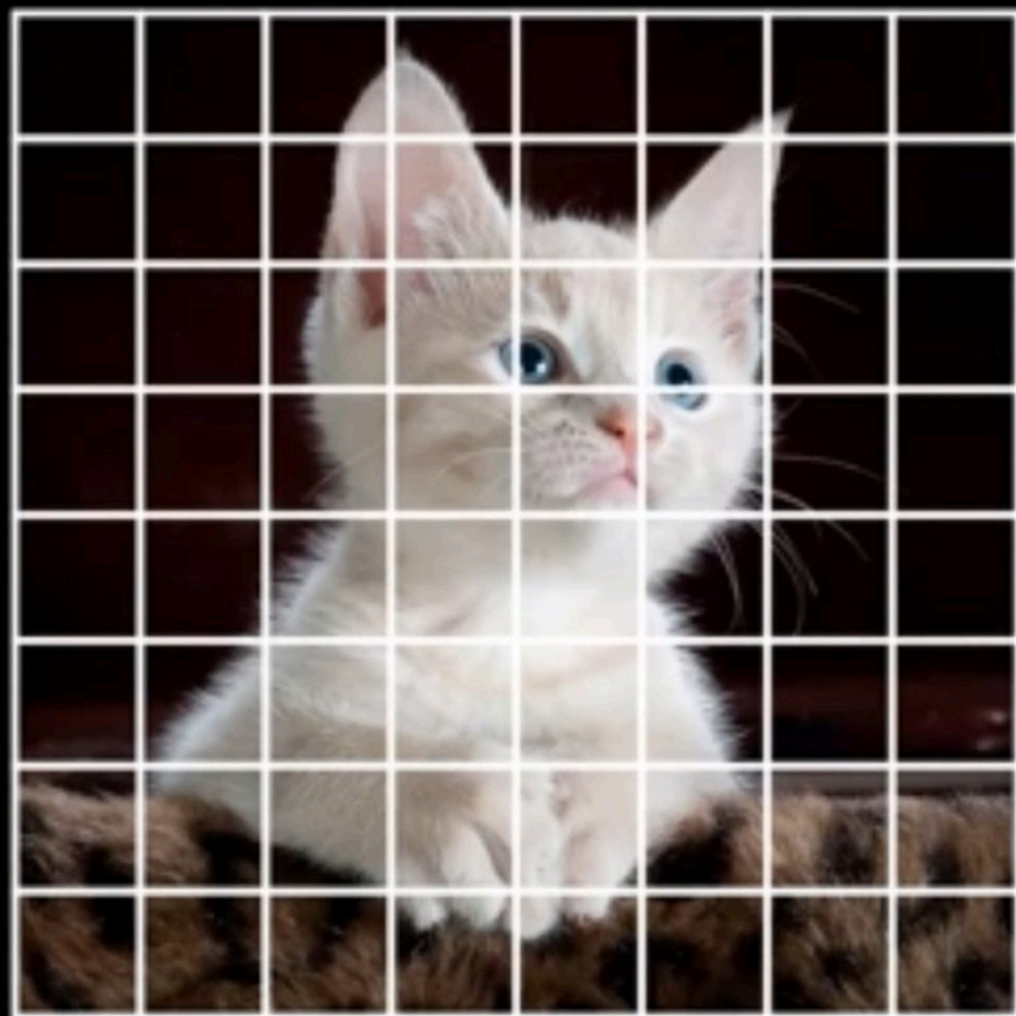
Artificial Intelligence is man-made.
It works with data and algorithms.

Vision Transformer(ViT)



81	28	25	95	179	61	234	318
92	3	98	243	14	149	245	45
106	284	99	187	71	232	153	199
188	174	65	153	30	44	203	152
112	234	240	39	121	24	34	114
230	65	239	39	234	284	151	25
74	145	222	14	312	85	145	117
87	184	189	223	136	217	109	85

Vision Transformer(ViT)



99	153	146	62	130	146	69	253
1	248	232	245	157	240	91	9
194	46	1	153	160	138	236	77
21	134	237	242	86	5	141	162
98	136	98	222	256	11	314	10
237	91	150	0	169	21	1	139
90	58	238	222	245	258	168	197
130	11	37	15	134	174	167	216

172	103	36	68	180	148	5	112
3	87	184	26	253	37	54	58
20	187	254	243	68	215	77	148
112	252	1	147	107	46	74	87
100	254	112	188	71	229	74	72
227	14	39	193	251	174	314	78
230	137	97	164	4	61	151	125
44	228	220	136	162	20	43	83

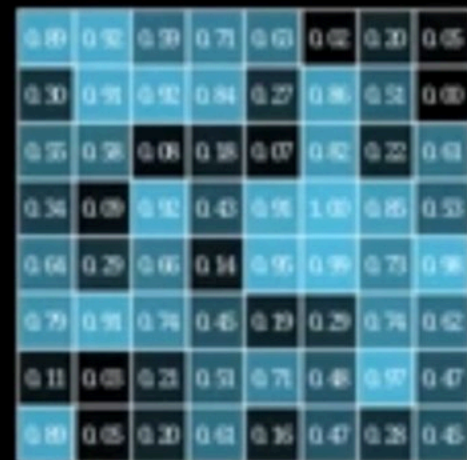
226	226	134	188	180	6	52	14
76	213	226	213	68	229	129	0
140	147	20	46	79	239	57	136
86	24	226	110	231	254	228	131
162	73	189	36	242	253	167	248
311	228	189	116	49	74	189	126
29	7	54	138	181	123	248	119
228	13	50	135	40	120	72	115

Vision Transformer(ViT)

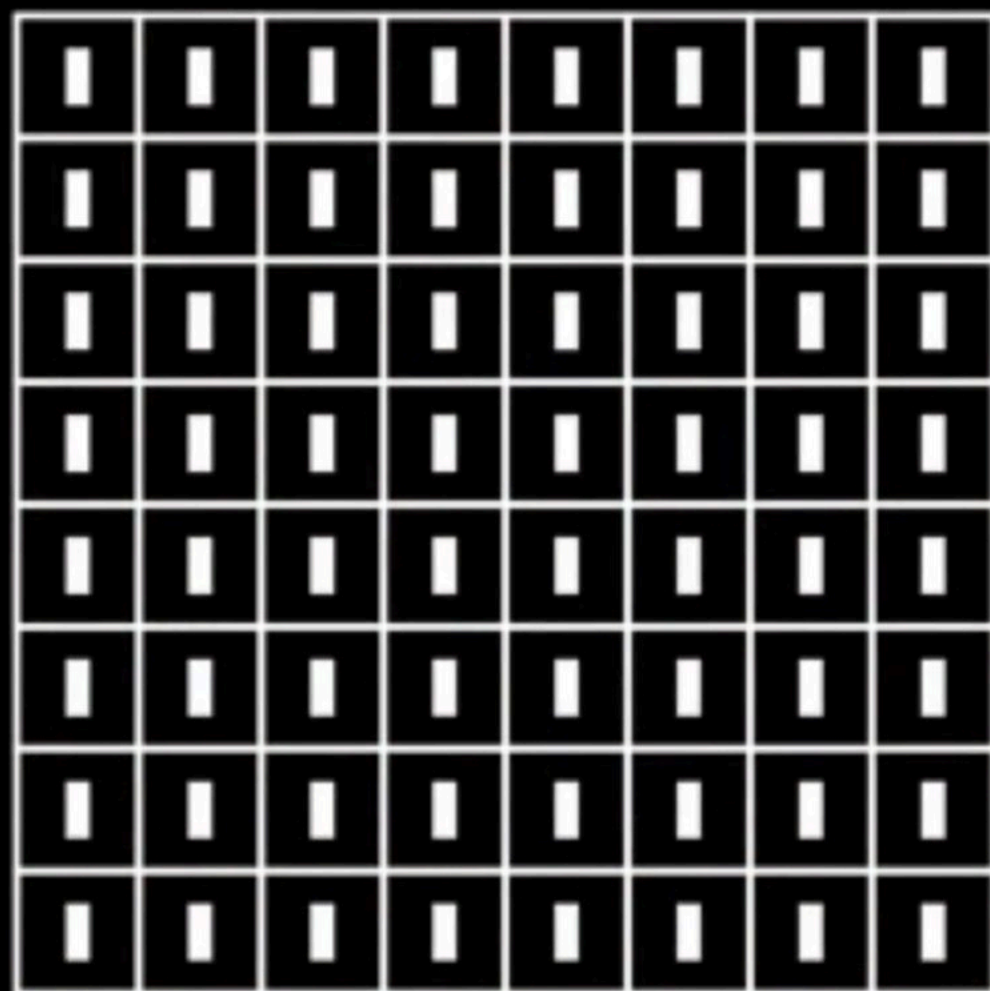


Patch size = $8 \times 8 = 64$

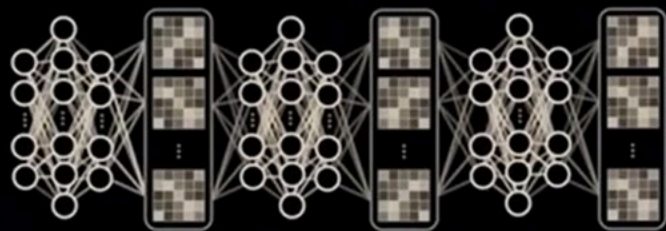
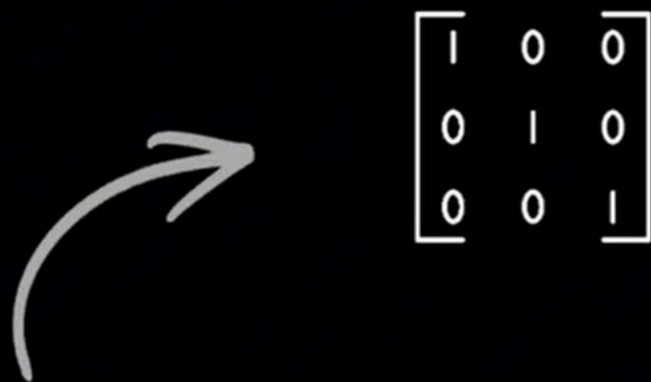
Total patches = $8 \times 8 = 64$



Vision Transformer(ViT)

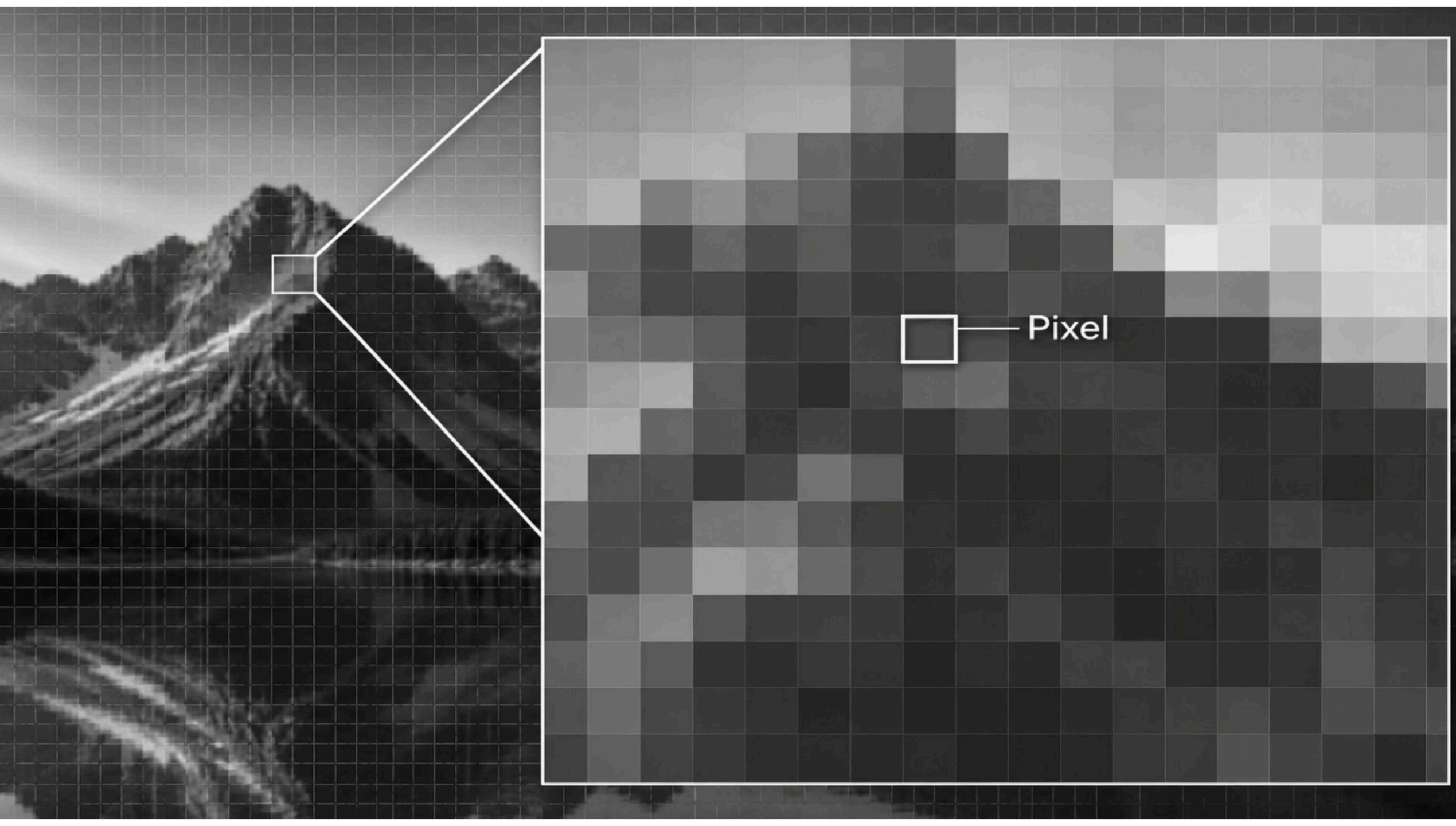


0.16
0.27
0.40
0.30
0.38
0.44
0.39
0.00
0.38
0.42
0.19
0.29
0.34
0.39
1.00
0.40
0.75
0.28
0.46
0.29
0.28
0.89
0.05
0.23
0.76
0.96



TRANSFORMER







0

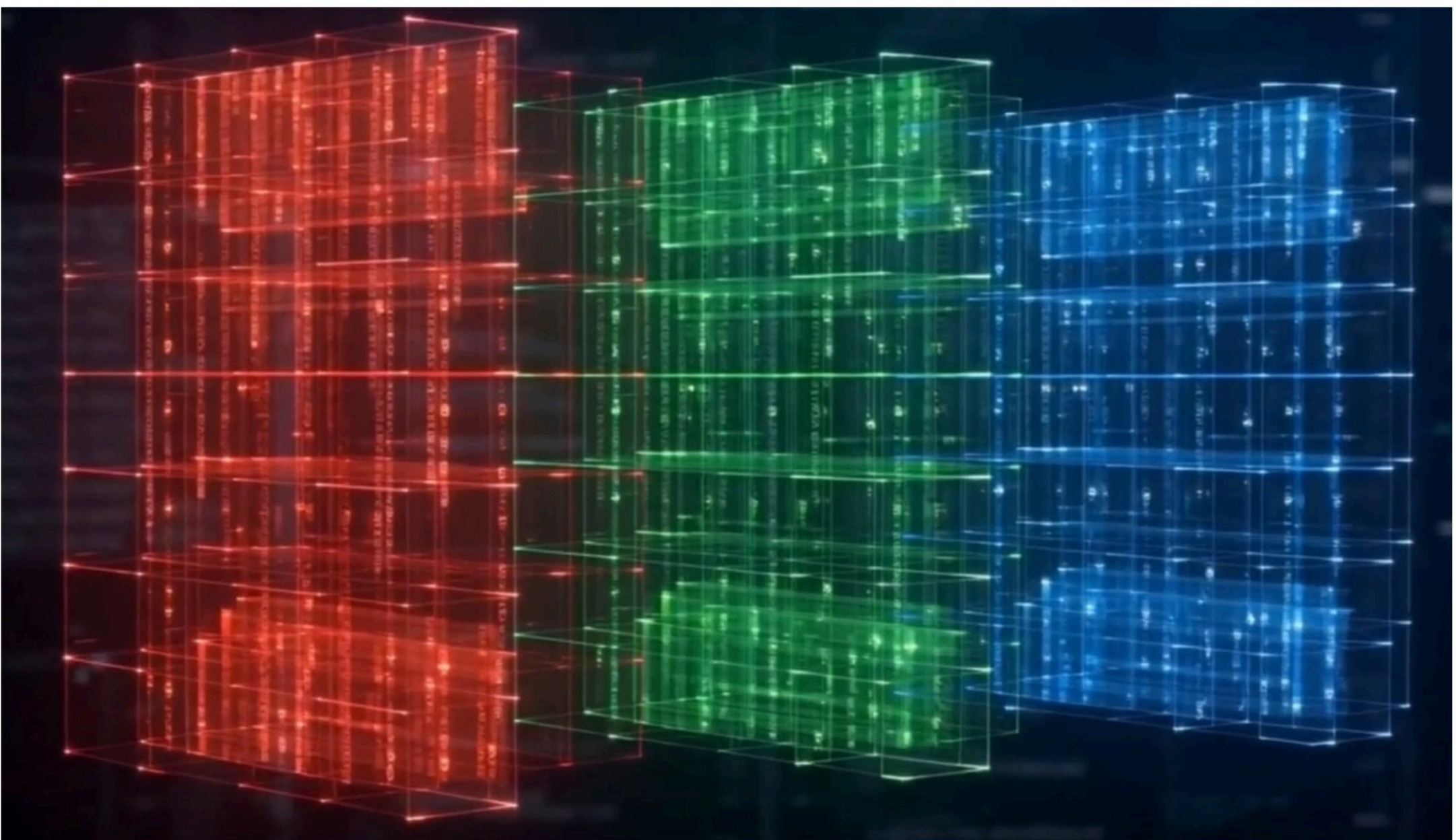
50

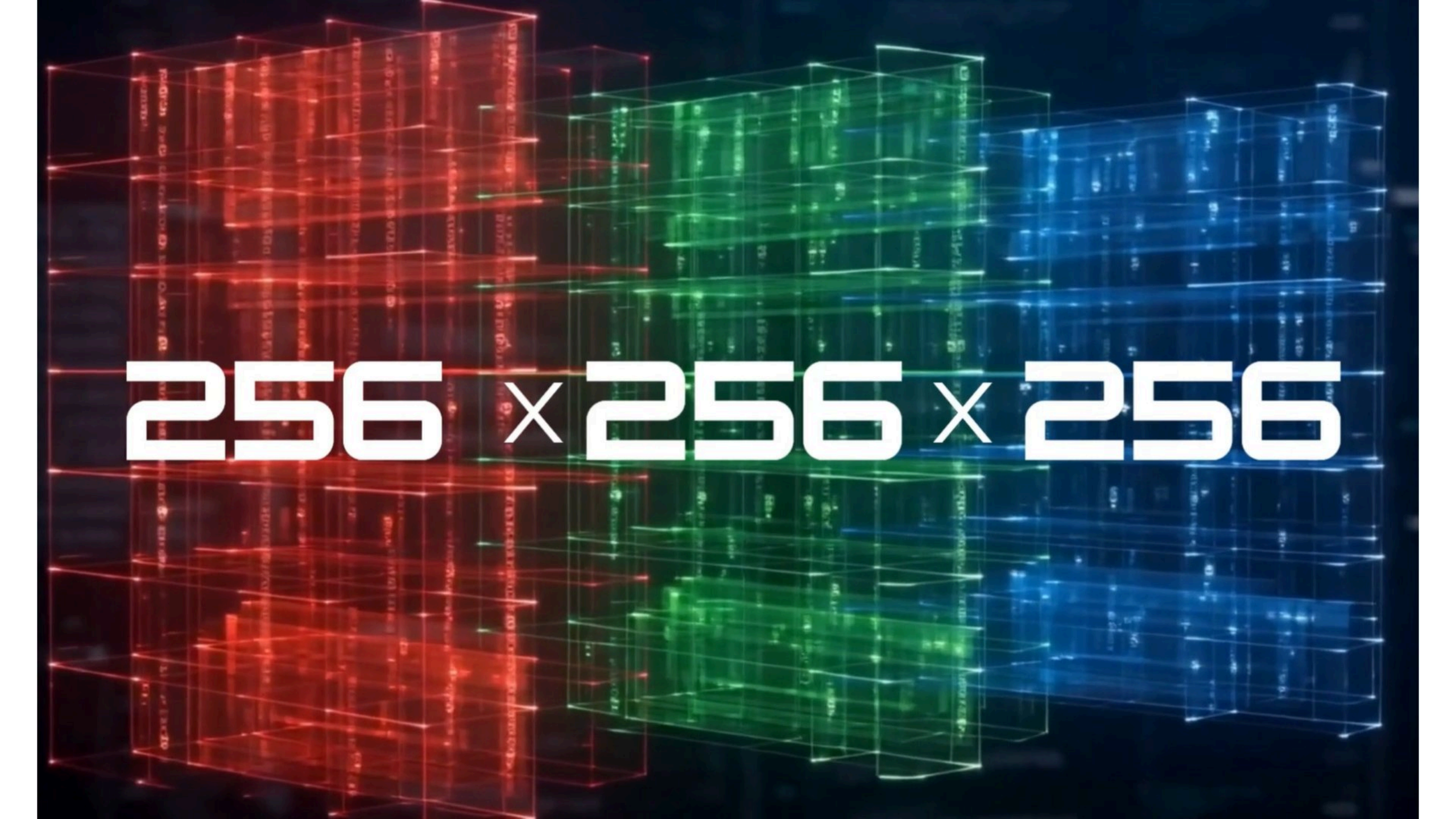
100

150

200

255





256 × 256 × 256



1677216
COLORS

Understanding Images as Tensors

Images might look like continuous scenes, but computers see them as numbers arranged in grids. This mathematical representation is the foundation of modern computer vision.



1. Grayscale Image: A 2D Grid

A grayscale image consists of a 2D grid of pixels. Each pixel holds a single intensity value between 0 (black) and 255 (white).



12	28	65	98	140	...
15	32	70	102	150	...
18	40	80	110	160	...
24	50	90	120	170	...
...	...	...	...	...	...



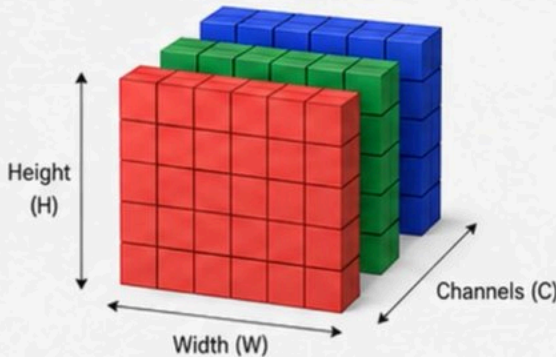
2. Color Image: Multiple Channels

A color image stores more information. Each pixel contains three values representing Red, Green, and Blue (RGB). These three channels combine to produce the final color.



3. From Grids to Tensors

Think of a color image as stacking multiple 2D grids together. The height and width define the spatial structure, and the channels add depth.



“In programming terms, we call 3D stack of grids a **Tensor**.”

A tensor is a multi-dimensional array of numbers. For a color image, the tensor shape is **(H, W, C)**, where **C = 3** for RGB.

Why It Matters?

Representing images as tensors allows algorithms and neural networks to process visual data efficiently. Almost every computer vision model works directly on these tensor representations.



Deep Learning Models



Object Detection

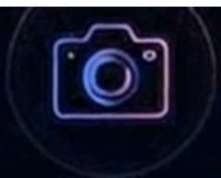


Image Segmentation

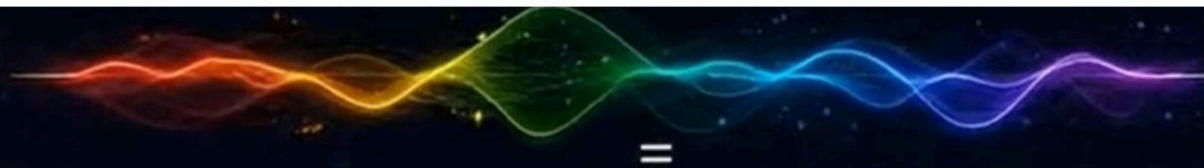


Image Classification

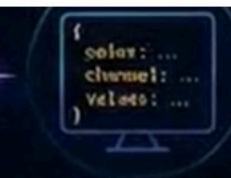




SAME IMAGE



DIFFERENT WAYS TO DESCRIBE THE COLORS



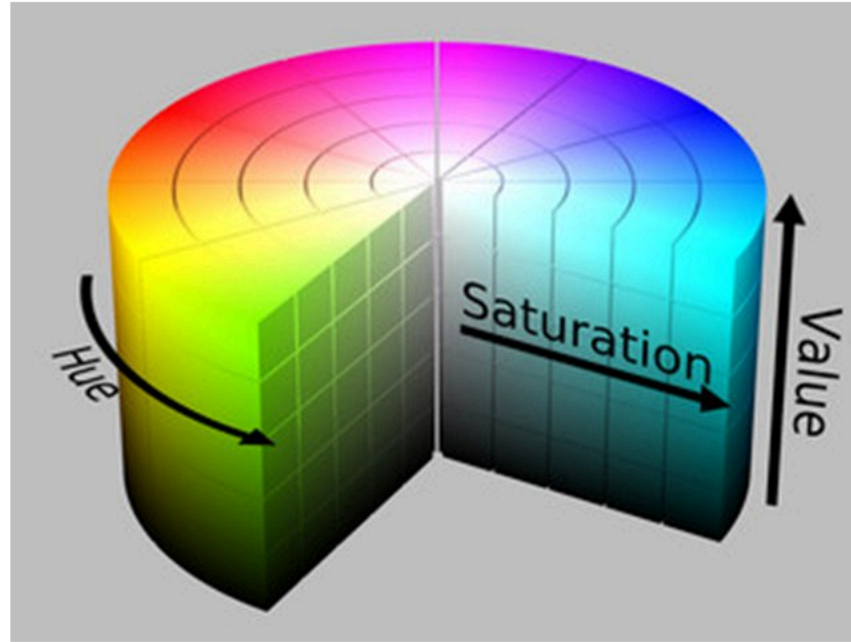
SAME INFORMATION



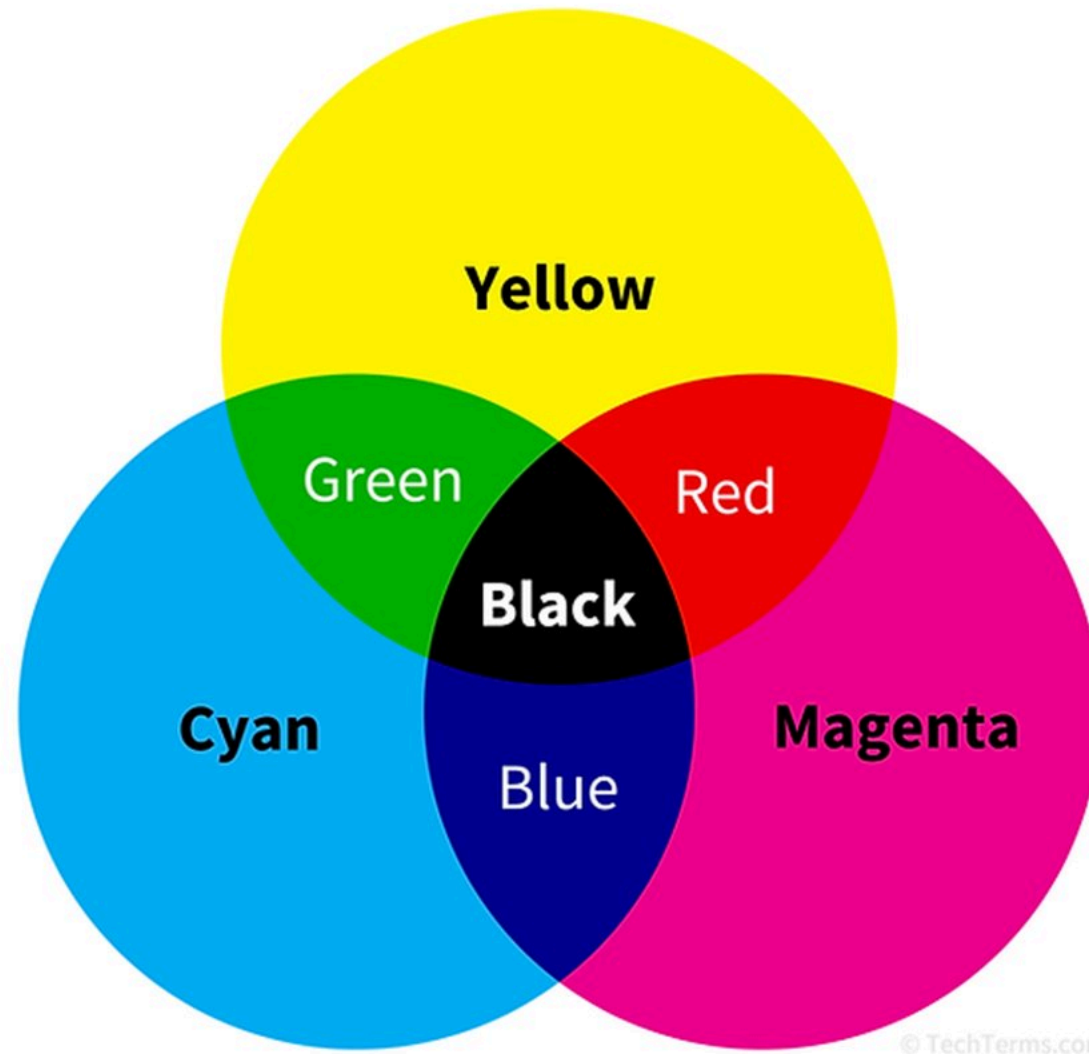
```
# Different representations, same image
image = load("mountain_scene.jpg")

rgb    = image.toRGB()
bgr    = image.toBGR()
hsv    = image.toHSV()
cmyk   = image.toCMYK()
lab    = image.toLAB()
gray   = image.toGRAY()
ycrcb  = image.toYCrCb()
```

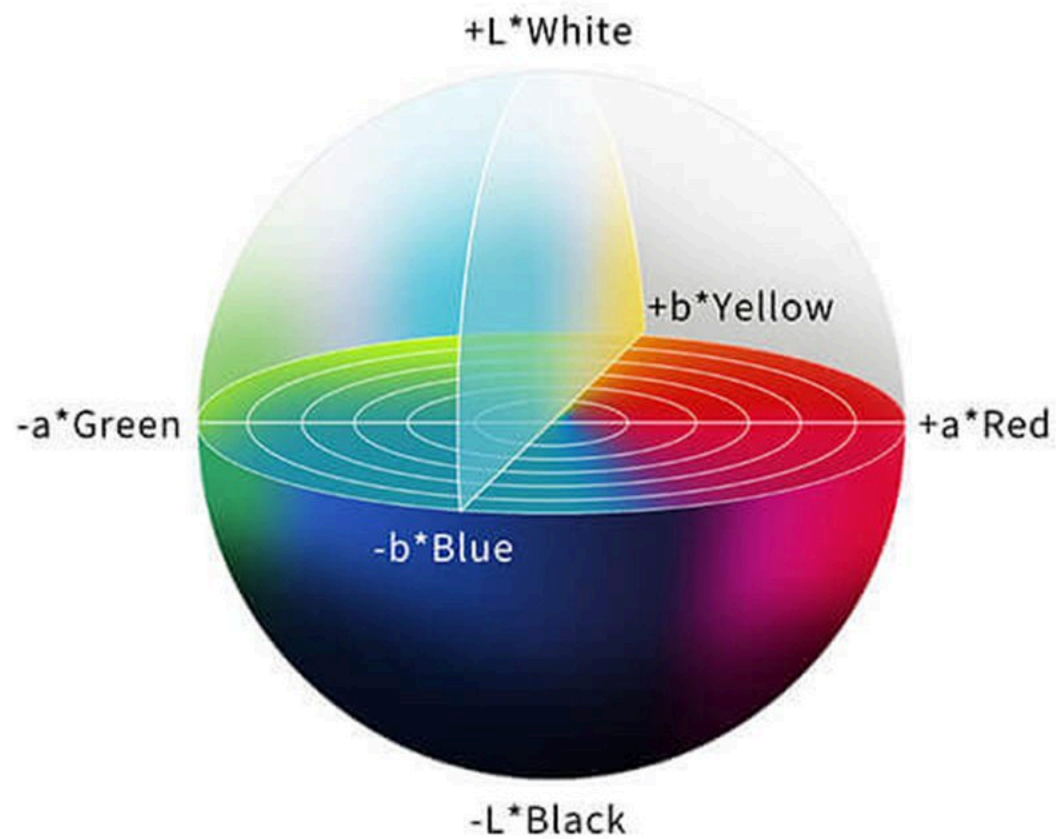
HSV



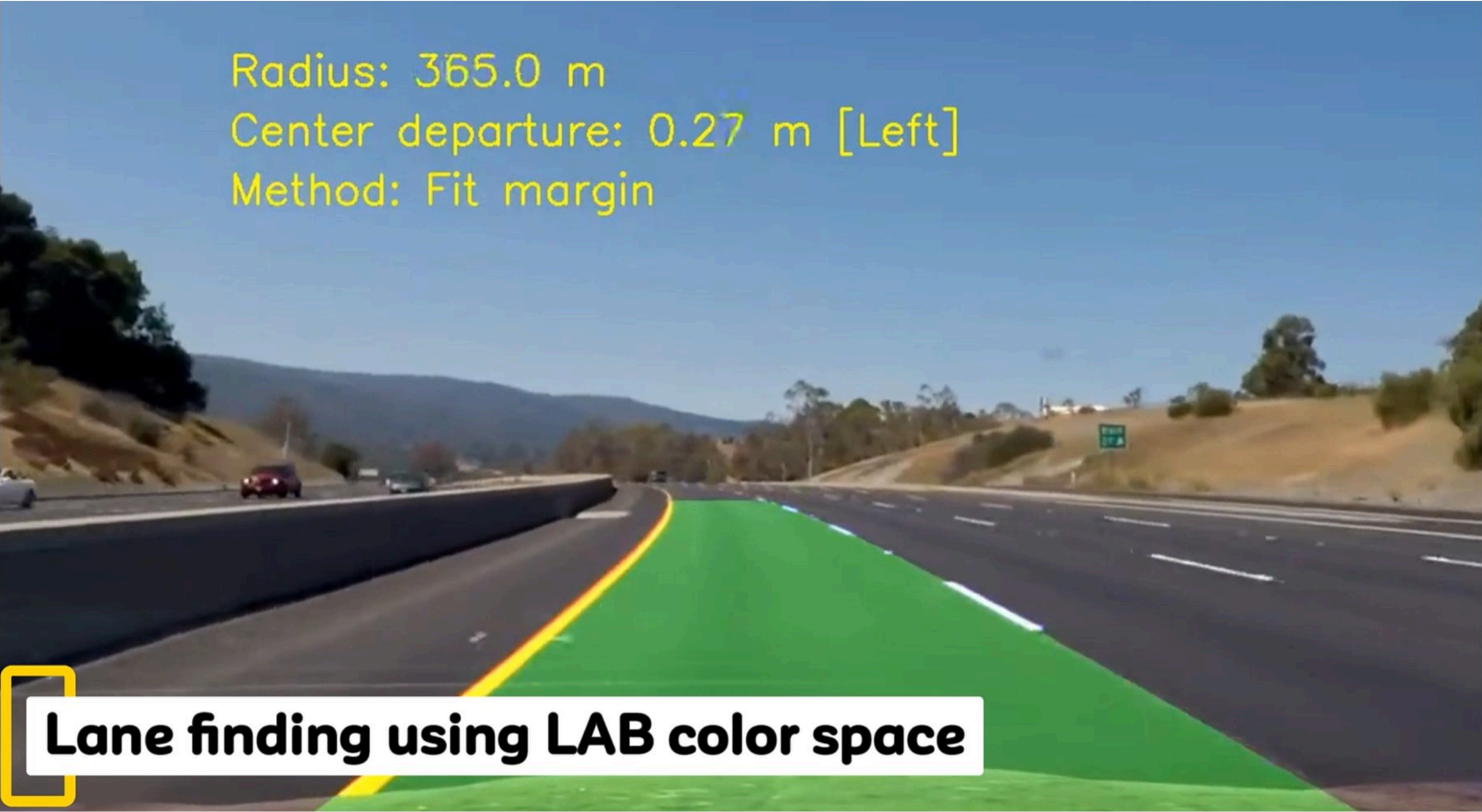
CMYK



LAB Color space



Radius: 365.0 m
Center departure: 0.27 m [Left]
Method: Fit margin



Lane finding using LAB color space



The image features three floating, rounded square panels with a metallic, reflective border. The left panel is dark gray and labeled 'Y'. The middle panel is blue and labeled 'Cb'. The right panel is orange and labeled 'Cr'. Each panel contains text describing its function. In the foreground, a wooden table holds a cup of coffee, a bowl of fruit, and a glass of milk. The background is a blurred outdoor scene with trees and a fence.

Y

– Brightness
(Luminancy)

Cb

– CB –
– Blue Color Difference

Cr

– Cr –
Red Color Difference

Filters

Slide a small **kernel** over every pixel and replace it with a weighted sum of its neighbours. Same image, different kernel, very different result.



Original
the input image

0	0	0
0	1	0
0	0	0

identity



Gaussian blur
smooth noise & detail

1	2	1
2	4	2
1	2	1

$\div 16$



Sharpen
boost edges & detail

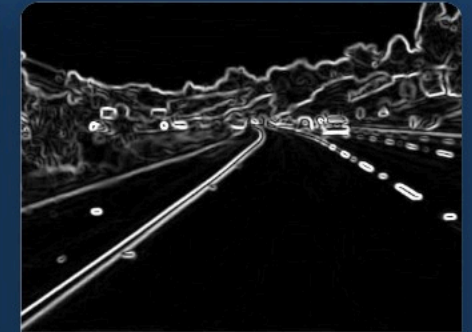
0	-1	0
-1	5	-1
0	-1	0

centre - neighbours



Median
remove salt-and-pepper noise

median of 5x5 non-linear



Sobel edges
find where brightness jumps

-1	0	1
-2	0	2
-1	0	1

x-gradient



**Blur
(Smooth)**

Reduce noise,
smooth details



**Averaging
Filter**

Reduce random
noise



**Sharpen
(Enhance)**

Enhance edges
and details



**Contrast
Enhance**

Improve contrast,
make it pop



**Edge
Detect**

Highlight edges
and shapes



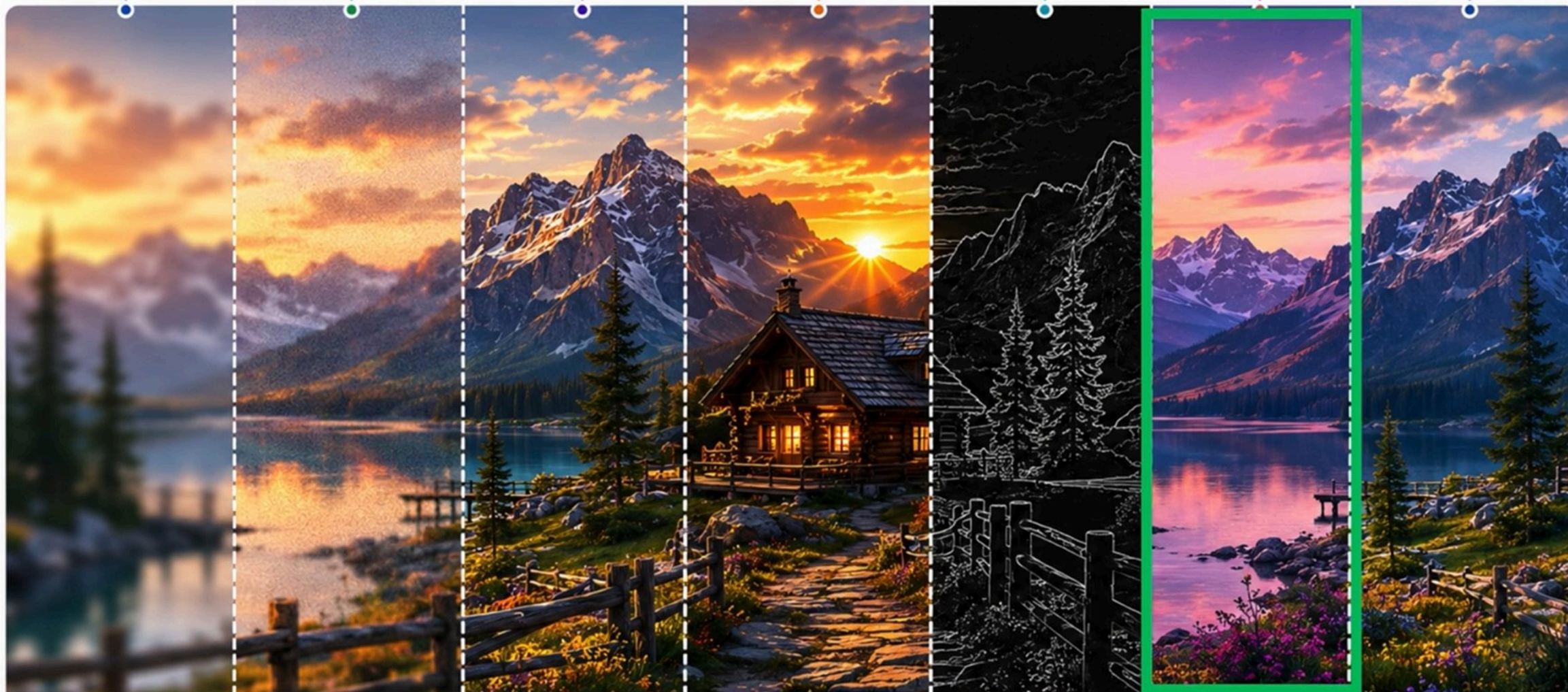
**Color
Filter**

Adjust color
tone/mood



**Denoise
(Median)**

Remove noise,
keep edges



AVERAGING

255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

FILTER

0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	0
0	0	0	0	255	0	0	0	0	0	0	0	255
0	0	0	0	255	0	0	0	0	0	0	0	0
0	0	0	0	255	0	0	0	0	0	0	0	255
0	0	0	0	255	0	0	0	0	0	0	0	0
255	255	255	255	255	255	255	255	255	255	255	255	255
0	0	0	0	255	0	0	0	0	0	0	0	0
0	0	0	0	255	0	0	0	0	0	0	0	255
0	0	0	0	255	0	0	0	0	0	0	0	255
0	0	0	0	255	0	0	0	0	0	0	0	255

+

255

1275

0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	
55	255	255	255	142	255	255	255	255	255	255	255	1275 ÷ 9 142
0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	
0	0	0	0	255	0	0	0	0	0	0	0	

Gaussian Blur



Median Blur

21	28	33	37	39	37	37	15
28	28	33	38	39	32	24	16
32	32	255	255	255	39	24	21
33	38	255	255	255	25	24	27
33	35	255	255	255	32	29	28
27	27	35	34	32	29	29	32
26	26	27	32	30	30	30	33
							38

Edge Detectors



```
image_filters.py x
import cv2

# -----
# Load Image
# -----
img = cv2.imread("tower_bridge.png")

if img is None:
    print("Error: Image not found!")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

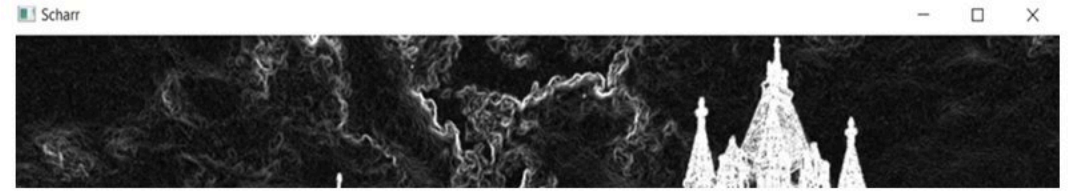
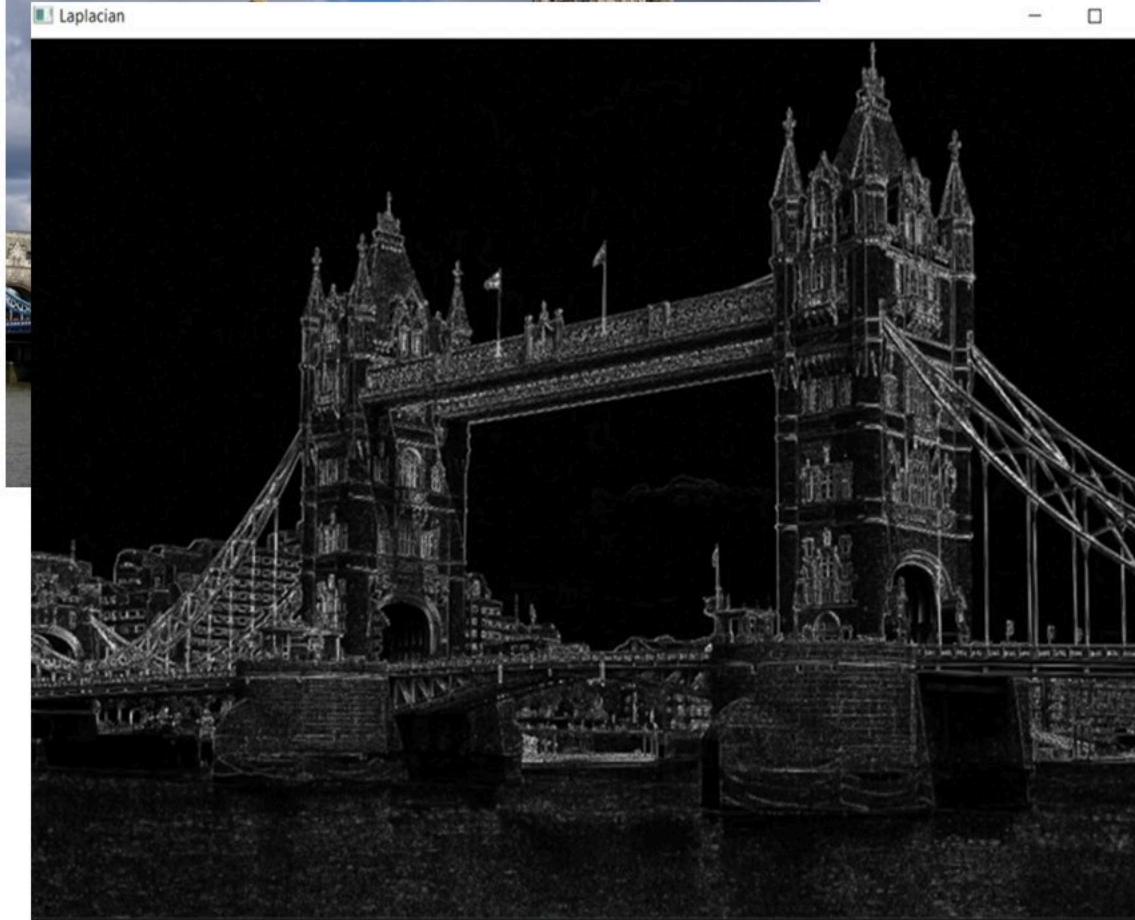
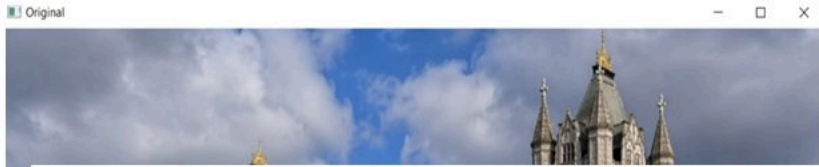
# -----
# Sobel Edge Detection
# -----
sobel_x = cv2.Sobel(gray, cv2.CV_64F, dx=1, dy=0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, dx=0, dy=1, ksize=3)
```

Run image_filters x

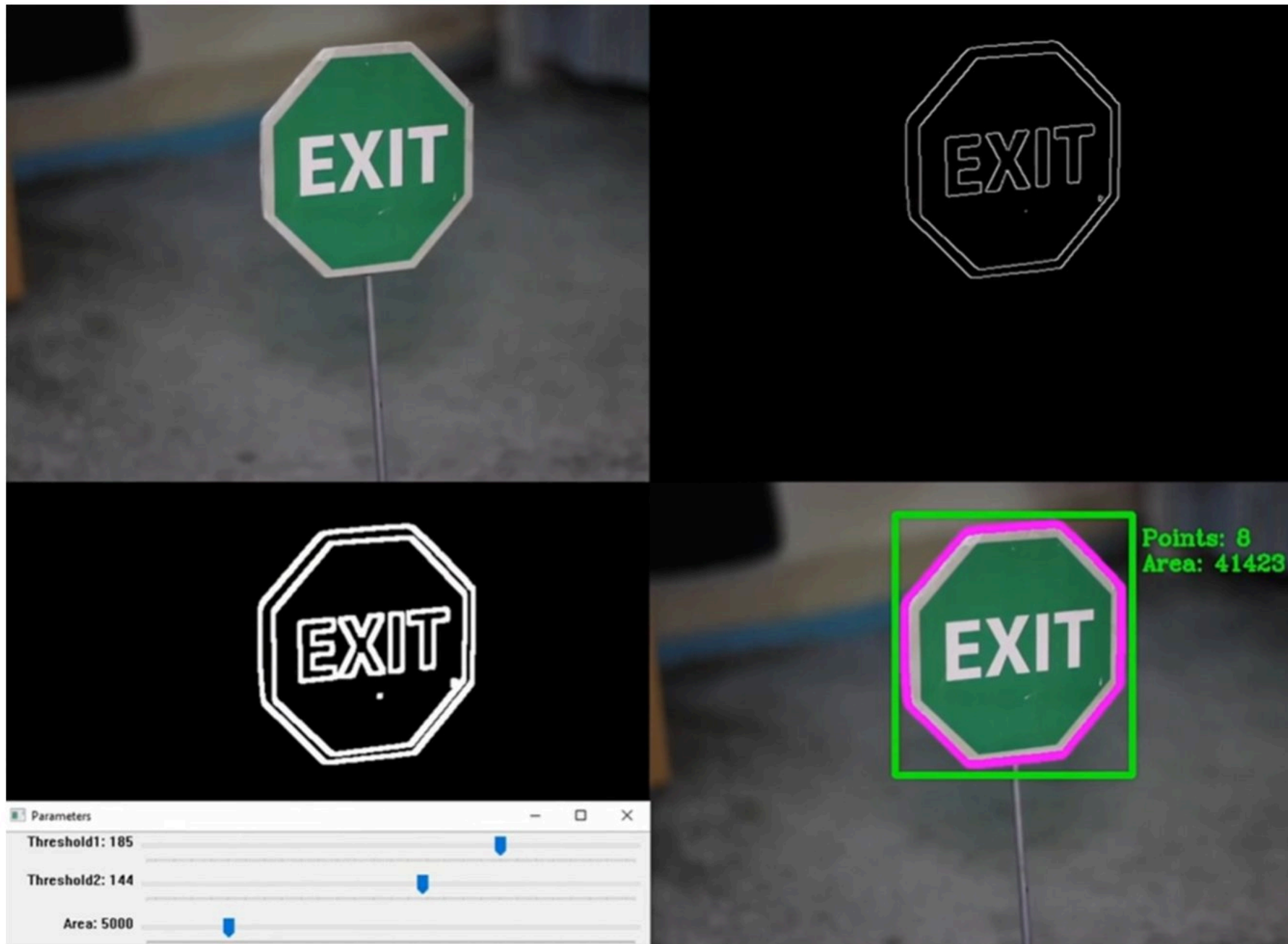
C:\Users\Recordings\Desktop\text_detect_validation\.venv\Scripts



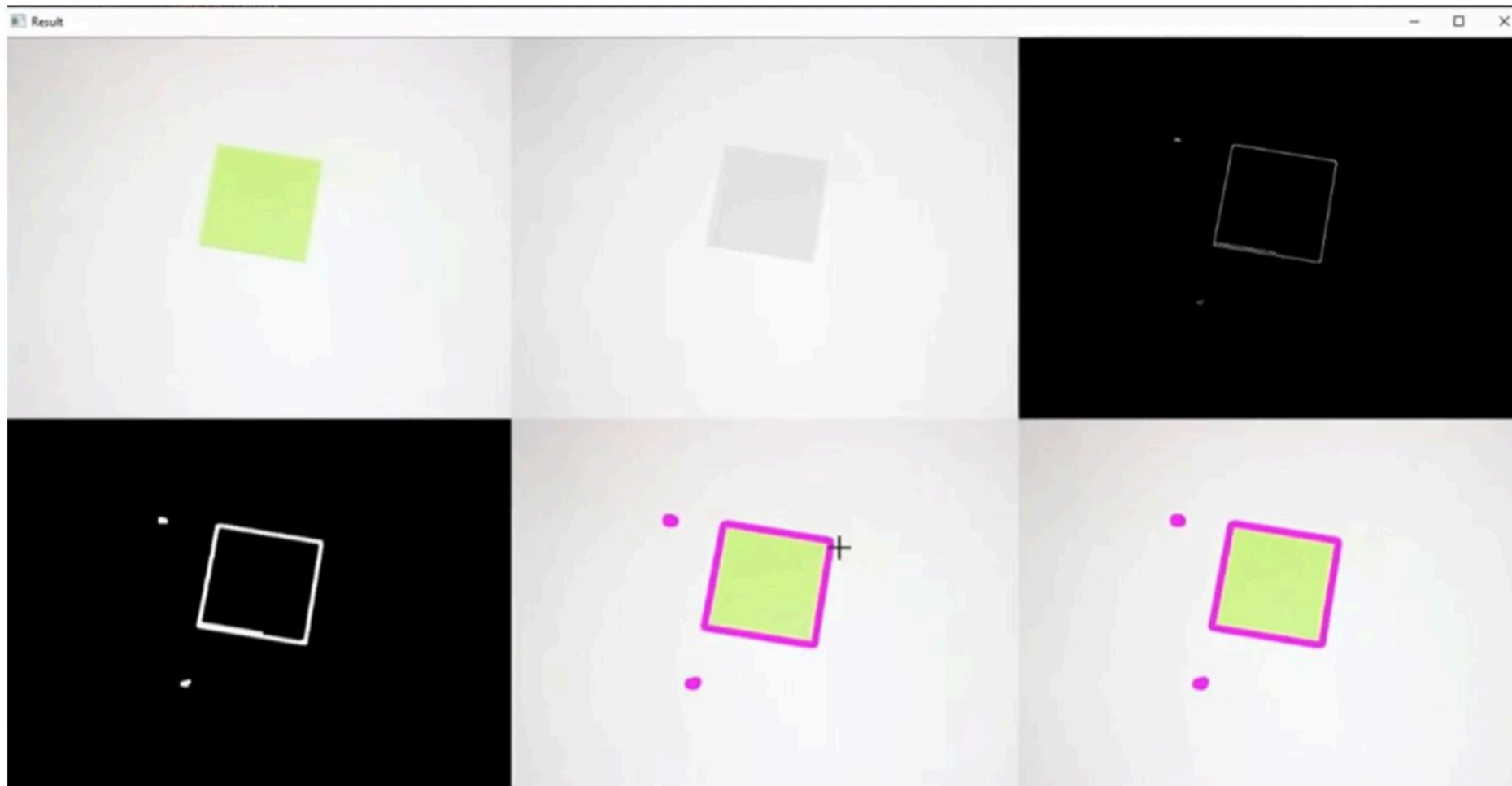
Edge Detectors



Shapes



Contours



1. Image



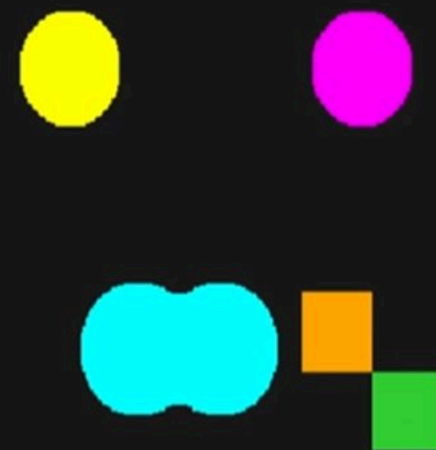
2. Prepared + noise



3. Cleaned (Morphology)

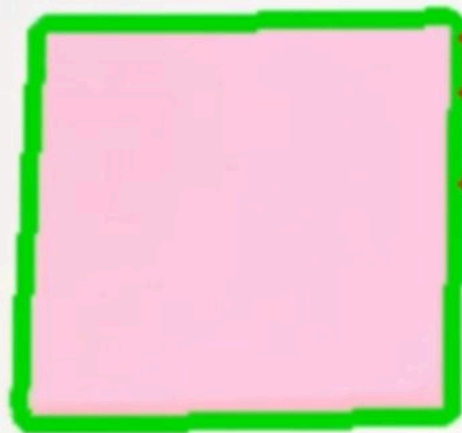


4. 5 objects (cleaned)

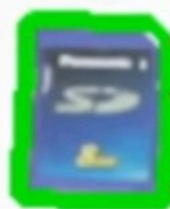


Without cleanup: 20 objects detected (noise + split counted as objects)

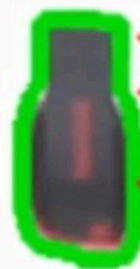
With cleanup: 5 objects detected -- the correct count



Points: 4
Area: 12853



Points: 5
Area: 1936



Points: 6
Area: 1760